



南开大学  
Nankai University

南 开 大 学

计 算 机 学 院

并行程序设计实验报告

---

## 多线程编程实验

---

林晖鹏

年级：2023 级

专业：计算机科学与技术

指导教师：王刚

2025 年 7 月 2 日

## 摘要

本文基于学校提供的服务器，实现了 GPU 并程序序设计实验，代码仓库地址：[link](#)。(相关代码在 GPU 服务器上。) 本文围绕多线程与 GPU 并行优化两个核心任务展开，针对口令猜测，从原始串拼接与概率计算两个方面，尝试引入 GPU 加速机制。首先，在基础任务中将字符串拼接过程并行化实现，详述了内存打包、CUDA 内核设计与数据传输策略，并通过 nvprof 性能分析工具评估优化效果。结果显示，由于计算密度较低、数据结构复杂以及频繁主机-设备间数据传输，GPU 优化效果不佳，甚至出现性能下降。随后在进阶任务中，进一步探索适合并行的计算模块，识别出概率计算为高频、重复、结构规整的操作，并设计 GPU 核函数批量处理所有 PT 的概率计算。在实现过程中遇到内存分配瓶颈，导致运行中断，提出可能为 `std::bad_alloc` 引发的问题。实验过程中，结合数据特性与 GPU 特性，提出合理的分析与优化建议，为今后 GPU 加速实践提供思路参考。

**关键字：**GPU 并行；CUDA；字符串拼接；概率计算；性能分析；内存管理

## 目录

|                             |           |
|-----------------------------|-----------|
| <b>一、 基础要求</b>              | <b>1</b>  |
| (一) GPU 并行优化简介 . . . . .    | 1         |
| (二) 基础代码 GPU 实现 . . . . .   | 1         |
| 1. 实现思路 . . . . .           | 1         |
| 2. 代码实现 . . . . .           | 1         |
| 3. 运行结果 . . . . .           | 4         |
| 4. 思考与讨论 . . . . .          | 6         |
| <b>二、 进阶要求</b>              | <b>6</b>  |
| (一) GPU 概率加速计算 . . . . .    | 6         |
| 1. 实现思路 . . . . .           | 6         |
| 2. 代码实现 . . . . .           | 7         |
| 3. 结果 . . . . .             | 9         |
| (二) GPU-hash 加速计算 . . . . . | 10        |
| <b>三、 总结</b>                | <b>10</b> |

## 一、 基础要求

### (一) GPU 并行优化简介

GPU 并行优化是指利用图形处理单元的并行计算能力,通过特定的编程模型(如 CUDA 或 OpenCL)对算法和数据处理进行优化,以显著提升计算性能。GPU 具有大量的计算核心,能够并行处理大规模数据,特别适用于矩阵运算、图像处理和科学计算等高计算密度任务。通过优化线程管理、内存访问模式和计算任务分配, GPU 并行优化可大幅减少计算时间,提升程序效率,广泛应用于高性能计算、机器学习和大数据处理等领域。

### (二) 基础代码 GPU 实现

#### 1. 实现思路

下面我们对原本代码中 TODO 的部分进行 GPU 并行,思路相当简单:

在 TODO 的这个地方,主要是拼接字符串,然后加入 guesses 中。由于 GPU 只能传入 C 风格的结构,而代码中很多数据结构都是 c++ 的 stl,无法直接传,将数据打包为连续的 char\* 字符串,然后传进去 GPU 中进行处理,最后在传回 CPU 主机中。

#### 2. 代码实现

1. 设计一个函数进行 GPU 封装处理:val 是传入要拼接的字符串,对应原本中的 a->ordered\_values, num 是要拼接的数量, guesses 是原本的 guesses 结果, guess 是拼接的字符串前缀。

```
1 void GPU(std::vector<std::string> &val, int num, std::vector<std::string>
   &guesses, const std::string &guess="");
```

2. 下面主要介绍 GPU 函数的流程:

(1) 先准备一些辅助变量的打包。有每个字符串在总字符串里面的偏移量、每个字符串的长度

```
1 const char* prefix = guess.c_str();
2 int prefix_len = guess.length();
3
4 // 为扁平化的字符串数据预先计算总长度
5 int values_total_len = 0;
6 int* h_lengths = new int[num];
7 int* h_offsets = new int[num];
8 int* h_output_offsets = new int[num];
9
10 int output_total_len = 0;
11
12 for (int i = 0; i < num; i++) {
13     h_lengths[i] = val[i].length();
14     h_offsets[i] = values_total_len;
15     h_output_offsets[i] = output_total_len;
16
17     values_total_len += h_lengths[i];
```

```

18         output_total_len += prefix_len + h_lengths[i] + 1; // +1 for null
           terminator
19     }

```

(2) 把所有待拼接的字符串扁平化压缩为一个 char\* 字符串, 用于传数据给 GPU。到这里, 所有要用的数据已经打包完成。

```

1     char* h_values_data = new char[values_total_len];
2
3     // 复制所有字符串数据到扁平化的数组中
4     for (int i = 0; i < num; i++) {
5         memcpy(h_values_data + h_offsets[i], val[i].c_str(), h_lengths[i]);
6     }

```

(3) 数据已经打包完成, 我们开始把数据传给 GPU 端。

```

1     // 一次性分配所有设备内存
2     char* d_prefix;
3     char* d_values_data;
4     int* d_lengths;
5     int* d_offsets;
6     int* d_output_offsets;
7     char* d_output;
8
9     cudaMalloc(&d_prefix, prefix_len + 1);
10    cudaMalloc(&d_values_data, values_total_len);
11    cudaMalloc(&d_lengths, num * sizeof(int));
12    cudaMalloc(&d_offsets, num * sizeof(int));
13    cudaMalloc(&d_output_offsets, num * sizeof(int));
14    cudaMalloc(&d_output, output_total_len * sizeof(char));
15
16    // 一次性复制所有数据到设备
17    cudaMemcpy(d_prefix, prefix, prefix_len + 1, cudaMemcpyHostToDevice);
18    cudaMemcpy(d_values_data, h_values_data, values_total_len,
               cudaMemcpyHostToDevice);
19    cudaMemcpy(d_lengths, h_lengths, num * sizeof(int),
               cudaMemcpyHostToDevice);
20    cudaMemcpy(d_offsets, h_offsets, num * sizeof(int),
               cudaMemcpyHostToDevice);
21    cudaMemcpy(d_output_offsets, h_output_offsets, num * sizeof(int),
               cudaMemcpyHostToDevice);

```

(3) 然后开始调用核函数来 GPU 执行。这里调用结束之后, 同步一下设备确保 GPU 执行完成

```

1     // 启动内核
2     int blockSize = 256;
3     int gridSize = (num + blockSize - 1) / blockSize;
4     processStringKernel<<<gridSize, blockSize>>>(d_prefix, prefix_len,
           d_values_data, d_lengths, d_offsets, num, d_output, d_output_offsets

```

```

    );
5    //同步设备
6    cudaDeviceSynchronize();

```

(4) 结束之后整理结果传到 guesses 中。然后释放内存 (略)。

```

1    // 将结果复制回主机
2    char* h_output = new char[output_total_len];
3    cudaMemcpy(h_output, d_output, output_total_len, cudaMemcpyDeviceToHost);
4    // 构建结果字符串
5    for (int i = 0; i < num; i++) {
6        guesses.push_back(std::string(h_output + h_output_offsets[i]));
7    }
8
9    // 更新全局计数
10   total_guesses += num;

```

3. 下面来解释一下核函数的设计:

我们传入扁平化的字符串, 并且传入了每个字符串的偏移量, 然后进行写入结果字符串, 最终传出结果

```

1    __global__ void processStringKernel(const char* prefix, int prefix_len,
2                                       const char* values_data, int* lengths, int*
3                                       offsets,
4                                       int num, char* output, int* output_offsets)
5    {
6        int idx = blockIdx.x * blockDim.x + threadIdx.x;
7
8        if (idx < num) {
9            int val_len = lengths[idx];
10           int val_offset = offsets[idx];
11           int result_len = prefix_len + val_len;
12           int out_offset = output_offsets[idx];
13           // 复制前缀字符串
14           for (int i = 0; i < prefix_len; i++) {
15               output[out_offset + i] = prefix[i];
16           }
17           // 复制值字符串
18           for (int i = 0; i < val_len; i++) {
19               output[out_offset + prefix_len + i] = values_data[val_offset + i];
20           }
21           // 添加终止符
22           output[out_offset + result_len] = '\0';
23     }
24 }

```

### 3. 运行结果

本节对比不同优化级别下 CPU 与 GPU 版本的性能，评估 GPU 并行优化的效果。表 1 展示了不同优化级别下 CPU 原版与 GPU 加速版本的猜测时间（Guess Time）及加速比。实验结果表明，GPU 加速版本的性能未达预期，不仅未能实现加速，反而在高优化级别下性能显著下降。

表 1: GPU-O0 加速与原版 Guess Time 对比（单位：秒）

| CPU 优化级别 | 版本     | Guess Time | 加速比  |
|----------|--------|------------|------|
| O0       | 原版     | 7.5328     | 1.00 |
|          | GPU 加速 | 8.87284    | 0.85 |
| O1       | 原版     | 0.431917   | 1.00 |
|          | GPU 加速 | 2.26515    | 0.19 |
| O2       | 原版     | 0.433922   | 1.00 |
|          | GPU 加速 | 2.22725    | 0.19 |

为了探究 GPU 加速效果不佳的原因，使用 `nvprof` 工具对 GPU 活动进行了详细分析。结果如表 2 所示，数据传输（包括主机到设备和设备到主机）占据了绝大部分时间，其中主机到设备拷贝（HtoD）占比高达 63.29%，而内核执行时间仅占 9.61%。这表明数据传输开销远超计算时间，导致 GPU 并行优化未达预期效果。优化级别对数据传输时间影响较小，因此在高优化级别下，GPU 版本与 CPU 版本的性能差距进一步拉大。

表 2: GPU 活动时间分布

| 操作类型           | 占比 (%) | 总时间 (ms) | 平均耗时 ( $\mu$ s) |
|----------------|--------|----------|-----------------|
| 主机到设备拷贝 (HtoD) | 63.29  | 17.867   | 6.925           |
| 设备到主机拷贝 (DtoH) | 27.11  | 7.653    | 14.831          |
| 内核执行           | 9.61   | 2.712    | 5.256           |

进一步对比了不同 GPU 优化级别下的性能表现，结果如表 3 所示。在 CPU 优化级别为 O1 和 O2 时，GPU 版本在 O1 和 O2 优化下相较于 O0 有所改进，加速比分别达到 1.58 和 1.49（O1 下）以及 1.49 和 1.46（O2 下）。然而，即便在最佳优化配置下，GPU 版本的猜测时间仍显著高于 CPU 原版，显示出数据传输开销对整体性能的限制作用。未来优化可重点考虑减少数据传输频率或优化内存访问模式，以提升 GPU 并行效率。

表 3: 不同 GPU 优化级别 Guess Time 对比（单位：秒）

| CPU 优化 | GPU 优化 | Guess Time | 相对 O0 加速比 |
|--------|--------|------------|-----------|
| O1     | O0     | 2.26515    | 1.00      |
|        | O1     | 1.43153    | 1.58      |
|        | O2     | 1.52316    | 1.49      |
| O2     | O0     | 2.22725    | 1.00      |
|        | O1     | 1.49623    | 1.49      |
|        | O2     | 1.52458    | 1.46      |

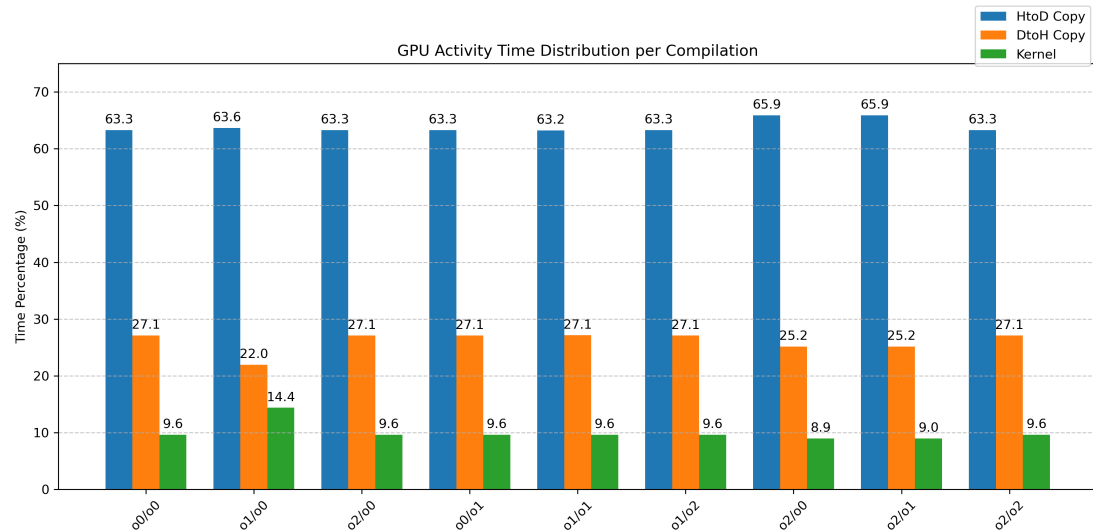


图 1: GPU 活动时间对比

为深入分析不同优化级别对 GPU 活动时间分布的影响，继续使用 **nvprof** 工具对各优化配置下的 GPU 活动进行了详细测量。表 4 展示了不同 CPU 和 GPU 优化级别组合下，主机到设备拷贝（HtoD）、设备到主机拷贝（DtoH）和内核执行的时间占比。结果显示，HtoD 拷贝始终占据主导地位，占比在 63.23% 至 65.90% 之间，表明数据传输开销在所有优化配置下均是性能瓶颈。优化级别对内核执行时间占比的提升有限，且在某些配置（如 O2/O0 和 O2/O1）下，HtoD 拷贝占比略有增加，进一步凸显数据传输对性能的限制。

表 4: GPU 活动时间占比

| 编译配置 (CPU/GPU) | HtoD Copy (%) | DtoH Copy (%) | Kernel (%) |
|----------------|---------------|---------------|------------|
| O0/O0          | 63.29         | 27.11         | 9.61       |
| O1/O0          | 63.65         | 21.96         | 14.39      |
| O2/O0          | 63.27         | 27.10         | 9.63       |
| O0/O1          | 63.27         | 27.10         | 9.62       |
| O1/O1          | 63.23         | 27.14         | 9.63       |
| O1/O2          | 63.29         | 27.09         | 9.63       |
| O2/O0          | 65.90         | 25.17         | 8.94       |
| O2/O1          | 65.89         | 25.16         | 8.96       |
| O2/O2          | 63.29         | 27.08         | 9.63       |

图 1 直观展示了不同优化级别下 GPU 活动时间分布的对比，进一步印证了数据传输（尤其是 HtoD 拷贝）在整体性能中的主导作用。优化级别对内核执行时间的改善效果有限，说明当前 GPU 并行实现受限于数据传输效率，而非计算能力。未来优化方向可包括减少数据传输频率、优化内存分配策略或采用异步传输技术，以降低数据拷贝开销，提升整体性能。

#### 4. 思考与讨论

实验结果表明,当前任务的 GPU 并行优化效果不佳,未能实现预期的加速目标。结合 **nvprof** 分析,数据传输开销(尤其是主机到设备拷贝)占据了运行时间的绝大部分,严重限制了 GPU 的并行优势。以下从任务特性和 GPU 加速适用性两方面进行分析:

GPU 加速的适用性取决于任务的计算密集程度和数据传输需求。对于计算密集型任务(如矩阵运算、深度学习模型训练或大规模科学计算),GPU 能够通过并行处理显著提升性能。然而,本实验的任务可能具有较低的计算复杂度或较小的数据规模,导致内核执行时间较短,难以抵消数据传输的高开销。表 2 显示,内核执行仅占总时间的 9.61%,而数据传输占比高达 90.40%,说明任务的并行化收益不足以弥补传输成本。

在本实验中,任务仅仅为拼接字符串,计算开销并不是很大,盲目并行加速反而适得其反。综上,当前任务由于数据传输开销过高,不适合直接应用 GPU 加速。未来可通过任务重构或算法优化,评估是否能提升计算密集度或减少数据传输需求,以充分发挥 GPU 并行优势。

## 二、进阶要求

### (一) GPU 概率加速计算

#### 1. 实现思路

通过上面的思考,再回来看进阶要求里面要求做的多 PT 并行优化,无非就是用 **gpu** 来处理由于多 PT 一次性生成的一对字符串的处理,但是字符串处理并不适合这里的 GPU 优化,一方面传输花销大,另一方面,原代码框架的数据结构大多都是用 **stl** 书写的,这里使得 **gpu** 传输的打包花销更大了。

所以我们回看这份代码,发现了一个可以 **gpu** 并行优化的地方:概率计算。

```
1 vector<PT> new_pts = priority.front().NewPTs();
2 for (PT pt : new_pts){
3     CalProb(pt);
4     ...
5 }
```

在原代码这里,是取出来每个概率然后在计算概率,但是计算概率是一次重复的乘除过程,这里我们完全可以一次行计算所有 **pt** 的概率,在进行后面的处理:

```
1 //vector<PT> new_pts = priority.front().NewPTs();
2 GPU_CalProb(new_pts, m)
3 for (PT pt : new_pts) CalProb(pt);
4 for (PT pt : new_pts){
5     ...//后续操作
6 }
```

所以这里**实现思路**就是 GPU 并行化 **CalProb** 这个函数:一次性先记录所有要乘的要除的数,然后传入 GPU,让 GPU 统一计算,然后在输出结果到主机。

由于这里不好把 **model** 类里面的东西放进 GPU 里面,比如下面这个代码,用到了很多 **m** 类对象里面的变量,不方便放进 GPU 里面,所以我们先用 CPU 把所有的要计算传入 GPU。

```
pt.prob* = m.letters[m.FindLetter(pt.content[index]).ordered_freqs[idx];
```



## 2. 代码实现

1. 首先是核函数，其实就是传入数据，然后在里面进行乘除计算。

```

1  __global__ void calculateProbabilityKernel(float* preterm_prob, float*
    multiplier, float* divisor, int* offsets, float* results, int num_pts) {
2  int pt_idx = blockIdx.x * blockDim.x + threadIdx.x;
3  if (pt_idx < num_pts) {
4      // 获取当前PT的起始和结束位置
5      int start = offsets[pt_idx];
6      int end = offsets[pt_idx + 1];
7      // 从preterm_prob开始计算
8      float probability = preterm_prob[pt_idx];
9      // 依次乘以每个乘数并除以每个除数
10     for (int i = start; i < end; i++) {
11         if (divisor[i] > 0) { // 防止除零错误
12             probability *= (multiplier[i] / divisor[i]);
13         }
14         else if( divisor[i] == 0) {
15             // 如果除数为0，直接跳过这个乘数
16             printf("出现除数为0的情况，跳过该乘数\n");
17             continue;
18         }
19     }
20     results[pt_idx] = probability;
21 }
22 }

```

2. 简单介绍 GPU\_CalProb 函数的流程（篇幅有点不够）：

- (1) 准备辅助变量：偏移量，每个 pt 要计算的乘数和除数的个数等等

```

1  int num_pts = new_pts.size();
2  float* preterm_prob = new float[num_pts];
3
4  // 每个pt有若干个乘数和除数
5  // 先统计一共有多少个乘数除数
6  int total_count = 0;
7  int *offsets = new int[num_pts+1];
8  for(int i = 0; i < num_pts; ++i) {
9      preterm_prob[i] = new_pts[i].prob;
10     offsets[i] = total_count;
11     total_count += new_pts[i].curr_indices.size();
12 }
13 offsets[num_pts] = total_count; // 最后一个PT的结束位置

```

- (2) 打包乘数和除数

```

1  float *multiplier = new float[total_count];
2  float *divisor = new float[total_count];
3  int k = 0;

```

```

4   for(int i = 0; i < num_pts; ++i) {
5       int index = 0;
6       for (int idx : new_pts[i].curr_indices)
7       {
8           auto &pt = new_pts[i];
9           if (pt.content[index].type == 1)
10          {
11              multiplier[k]= m.letters[m.FindLetter(pt.content[index])].
12                  ordered_freqs[idx];
13              divisor [k]= m.letters[m.FindLetter(pt.content[index])].total_freq;
14              k++;
15          }
16          else if (pt.content[index].type == 2)
17          {
18              multiplier[k] = m.digits[m.FindDigit(pt.content[index])].
19                  ordered_freqs[idx];
20              divisor[k] = m.digits[m.FindDigit(pt.content[index])].total_freq;
21              k++;
22          }
23          else if (pt.content[index].type == 3)
24          {
25              multiplier[k] = m.symbols[m.FindSymbol(pt.content[index])].
26                  ordered_freqs[idx];
27              divisor[k] = m.symbols[m.FindSymbol(pt.content[index])].total_freq;
28              k++;
29          }
30          index += 1;
31      }
32  }

```

### (3) 分配内存、调用核函数、赋值结果

```

1   //分配内存,并拷贝传入
2   ...
3
4   // 设置CUDA核函数的执行配置
5   int blockSize = 256; // 每个块的线程数
6   int numBlocks = (num_pts + blockSize - 1) / blockSize; // 计算需要的块数
7   // 调用CUDA核函数
8   calculateProbabilityKernel<<<numBlocks, blockSize>>>(d_preterm_prob,
9       d_multiplier, d_divisor,
10                                     d_offsets, d_results
11                                     , num_pts);
12
13   // 检查CUDA错误
14   cudaError_t err = cudaGetLastError();
15   if (err != cudaSuccess) {
16       std::cerr << "CUDA error: " << cudaGetErrorString(err) << std::endl;
17       return;
18   }

```

```

16 //传结果回主机
17 ...
18 // 将结果存储回PT对象
19 for (int i = 0; i < num_pts; ++i) {
20     new_pts[i].prob = results[i];
21 }
22 //释放内存
23 ...

```

- (a) **概率数组初始化：**为每个 PT 分配并初始化概率数组 `preterm_prob`。
- (b) **统计偏移和总元素数量：**遍历所有 PT，计算每个 PT 对应的乘数和除数的数量，并生成偏移数组 `offsets` 记录各 PT 的起始位置。
- (c) **填充乘数和除数数组：**根据 PT 中符号的类型（字母、数字、符号），查询模型 `m`，分别填充 `multiplier` 和 `divisor` 数组。
- (d) **GPU 内存分配：**在 GPU 设备上分配内存，包括：
  - 初始概率数组 (`d_preterm_prob`)
  - 乘数数组 (`d_multiplier`)
  - 除数数组 (`d_divisor`)
  - 偏移数组 (`d_offsets`)
  - 结果数组 (`d_results`)
- (e) **数据传输：**将所有数组从主机内存复制到设备内存。
- (f) **核函数配置：**根据数据量计算 CUDA 核函数的执行配置（块数和线程数）。
- (g) **核函数调用：**调用 `calculateProbabilityKernel` 在 GPU 上并行执行概率计算。
- (h) **结果拷回：**将 GPU 计算的概率结果复制回 CPU。
- (i) **更新 PT：**将每个 PT 对象的概率字段更新为 GPU 计算结果。
- (j) **内存释放：**释放 GPU 和 CPU 中分配的所有内存。

### 3. 结果

很遗憾，我这里尽力了，我在这里尝试了很久，但是一直在报错一个地方：

```

1  Guesses generated: 559924
2  GPU CalProb start
3  Guesses generated: 660105
4  GPU CalProb start
5  terminate called after throwing an instance of 'std::bad_alloc'
6     what():  std::bad_alloc
7  test.sh: line 3: 712374 Aborted                  (core dumped) ./test.exe

```

ai 说这个一般是主机部分由于 `new` 的内存太大的原因导致的。但是我尝试了**分批处理**，每次处理一小批，甚至是 PT 概率分批开始痛 GPU 分批一致，已经小到每次 1000 个 PT 的概率计算，但是仍然报错这个错误。而且我是在 `.cu` 文件里面实现的这个 GPU 加速的函数（为了方便 `gpu` 编译），无法打印 `cout` 和 `printf`，所以我 `debug` 难上加难。而且这里不是没有计算，而是应该是算到某一次特别大导致无法 `new` 内存了导致报错。

## (二) GPU-hash 加速计算

结合前面的思考，这里还有一个很适合加速的地方是这里的 hash 运算，这里大规模计算十分适合 GPU 加速，但是由于篇幅和时间问题，没有时间做了。

## 三、 总结

本次实验围绕字符串拼接与概率计算两个典型任务，探索了 GPU 并行在不同类型工作负载中的适用性。实验结果表明，GPU 并非对所有任务都能带来加速收益。对于字符串拼接类计算，受限于 GPU 传输代价大、STL 结构转化开销高等问题，实际加速效果不理想，甚至出现负加速。在此基础上，转而挖掘任务中计算密集的环节，选取 PT 概率计算模块进行并行处理，虽然在技术实现上取得一定进展，但仍遇到内存管理瓶颈，说明复杂数据处理中的 GPU 加速仍需更细致的任务划分和更谨慎的资源分配策略。

本实验也反映了 GPU 优化的基本准则：适合 GPU 处理的任务应具备高度并行性、计算密集型和低数据传输需求的特征。今后在程序优化中，应从任务建模层面出发，结合数据依赖关系与并行性分析，合理划分并行子任务，并重视数据打包与资源控制策略，以发挥 GPU 最大潜能。同时，也可考虑混合并行、多线程与 GPU 协同等更具弹性的优化策略，从而提升整体程序性能。

结合本次实验过程与结果，提出以下几点 GPU 并行优化建议：

1. **优先选择计算密集型任务进行 GPU 并行。**对于数据处理与字符串拼接等计算强度较低、数据结构复杂的任务，GPU 并不一定能带来性能优势，甚至可能因传输与打包开销导致性能下降。
2. **尽可能减少主机与设备之间的数据传输。**利用异步数据传输、合并内存访问或持久化数据驻留策略，可显著降低数据传输带来的时间消耗。
3. **提前设计好数据结构与内存布局。**使用适合 GPU 的扁平化结构替代 STL 等复杂对象，同时控制单次分配的数据量，避免出现 `std::bad_alloc` 等内存溢出问题。
4. **任务分批处理与流水线设计。**对于大规模数据并行任务，可采用分批处理 (batching) 与流式执行方式，避免一次性占用大量内存资源。
5. **强化调试与分析手段。**建议使用 `cuda-gdb`、`printf` 宏、`thrust` 辅助工具等手段提升 GPU 代码可调试性，配合 `nvprof` 或 `nsight` 工具进行性能瓶颈分析。
6. **探索混合并行与任务重构。**对于不适合 GPU 执行的任务，可考虑使用多线程并行（如 OpenMP）与 GPU 协同处理，通过合理分工提升整体效率。